CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

KIẾN TRÚC KAFKA VÀ RABBITMQ

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Kafka and RabbitMQ Architectures

Kafka Components:Topic, Partitions, Offset and Replication Factor

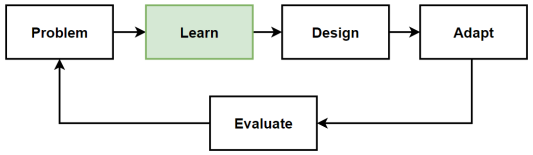
Apache Kafka Cluster Architecture:Kafka Brokers and Zookeeper

Apache Kafka Core APIs:Producer, Consumer, Streams and Connect API

RabbitMQ Components: Producer, Queue, Consumer, Message, Exchange, Binding

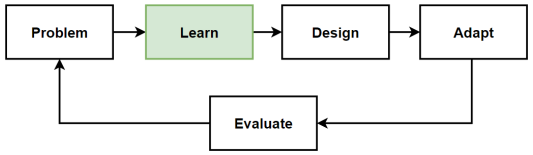
RabbitMQ Exchange Types

What is Apache Kafka ?



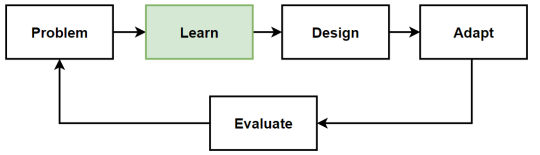
- Apache Kafka is the most popular **open-source event streaming** platforms.
- Designed for **horizontally scalable**, **distributed**, and **fault-tolerant**
- Distributed **publish-subscribe event streaming** platform
- Publishers **send** messages to **topics**, **subscriber** of a **topic** receives all the messages published to the topic.
- **Event-driven Architecture:** Kafka is used as an event router, and the microservices publish and subscribe to the events.
- **Distributed working**, and it provides a **horizontal scalable** system.
- Built on top of **ZooKeeper synchronization** service.
- **Key Components:** Topics, Partitions, Brokers, Producer, Consumer, Zookeeper

Apache Kafka Benefits

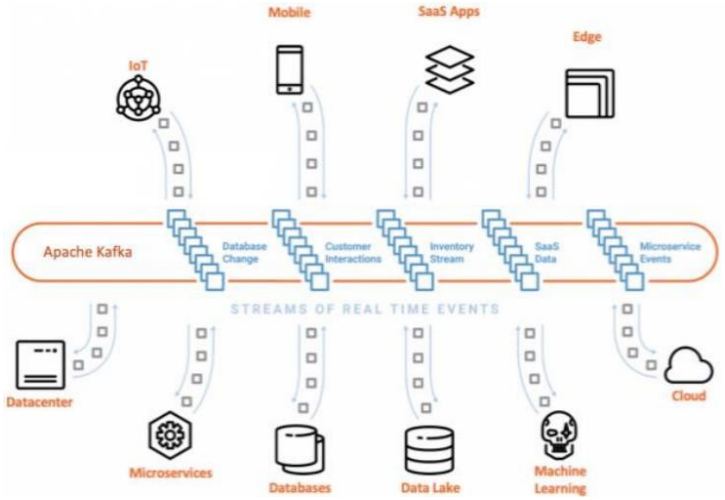


- **Reliability**
  Kafka is distributed, partitioned, replicated and fault tolerance. So it is also High Availability.
- **Scalability**
  Since its distributed architecture, Kafka scales easily without any down time.
- **Durability**
  Kafka uses Distributed commit log. This persists to events on disk as log commit very fast. Its durable for infinitive or a given parameter days or weeks.
- **Performance**
  Kafka has high throughput for both publishing and subscribing messages. It is distributed event streaming platform capable of handling trillions of events a day.
- Kafka has better throughput, built-in partitioning, replication, and fault-tolerance architecture.

Apache Kafka Use Cases

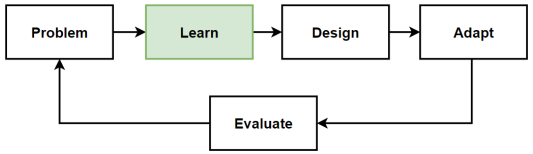


- **Messaging**
  Message brokers are used to decouple services from each other. In event-driven architecture, Kafka is used as an event router, and microservices publish and subscribe to the events.
- **Metrics**
  Kafka is often used for operational monitoring data. Aggregating statistics from distributed applications to produce centralized feeds of operational data.
- **Log Aggregation**
  Log aggregation solution that collect logs from multiple services. Log aggregation collects physical log files servers and puts them in a central place for processing.
- **Stream Processing**
  Kafka process data in processing pipelines consisting of multiple stages. Storm and Spark Streaming read data from a topic and processes it.

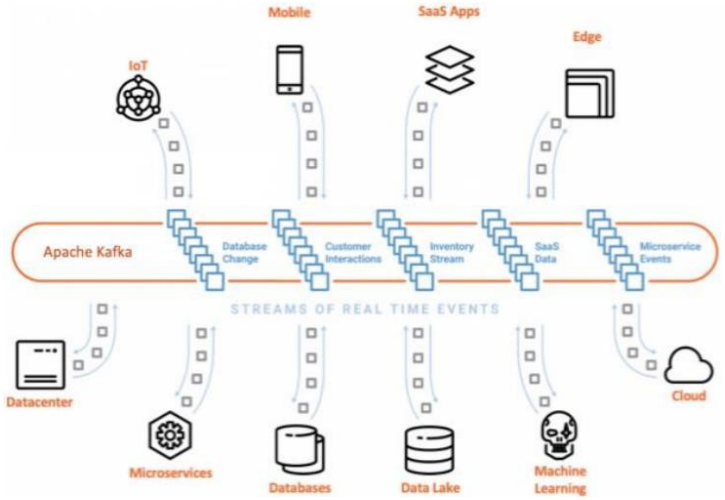


- Global-scale
- Real-time
- Persistent Storage
- Stream Processing

Apache Kafka Use Cases

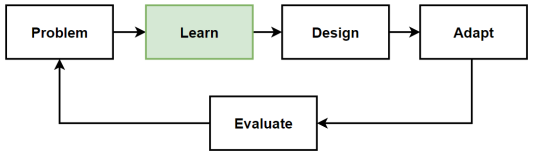


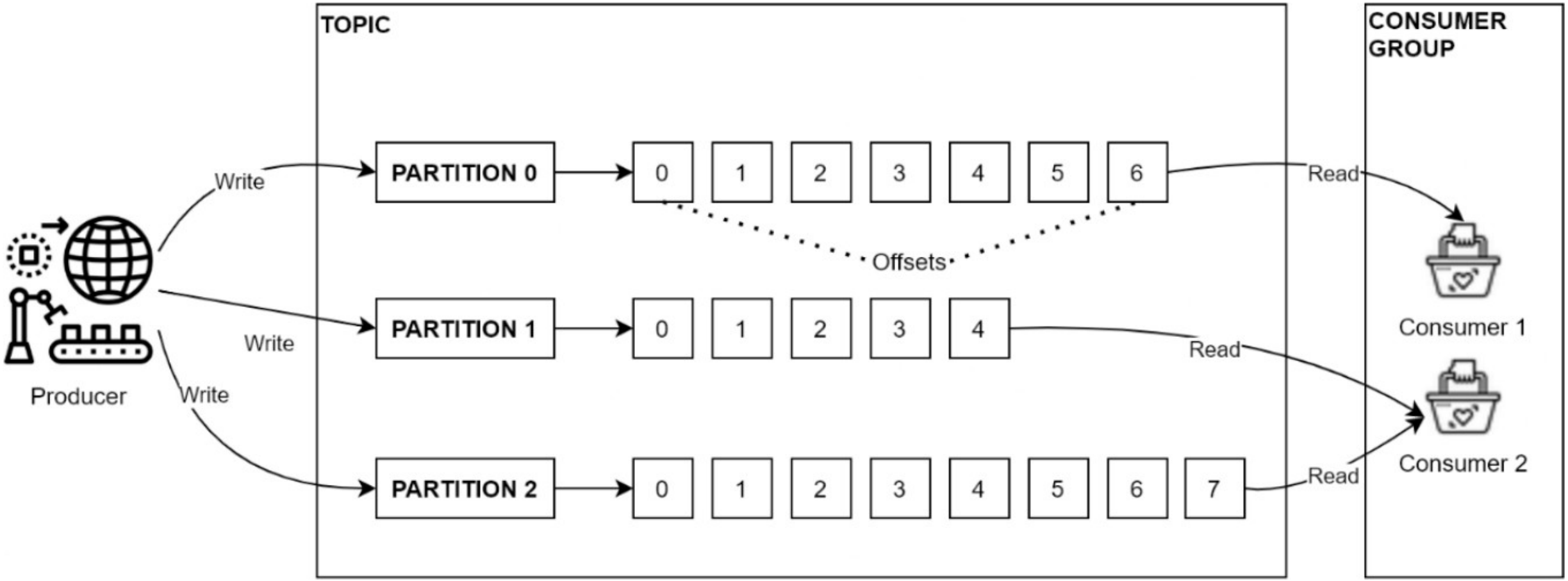
- **Website Activity Tracking**
  Kafka follows the user activity tracking pipeline as a set of real-time publish-subscribe feeds.
- **Event Sourcing**
  Kafka is using with the event-driven architecture, so in that cases also can be using for event storing and event sourcing.



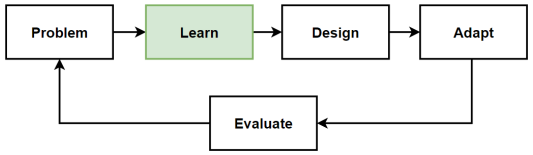
- Global-scale
- Real-time
- Persistent Storage
- Stream Processing

Kafka Components:
Topic, Partitions, Offset and Replication Factor

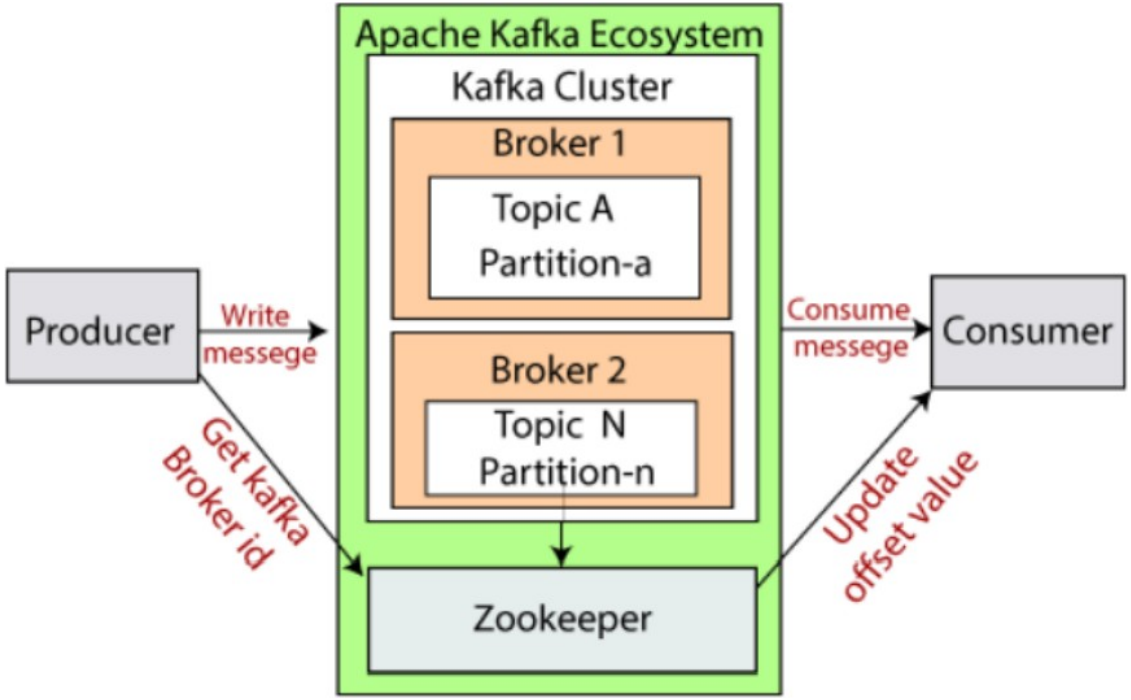




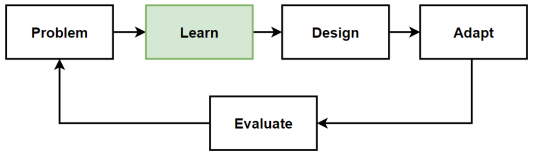
Apache Kafka Cluster Architecture: Kafka Brokers - Kafka Cluster



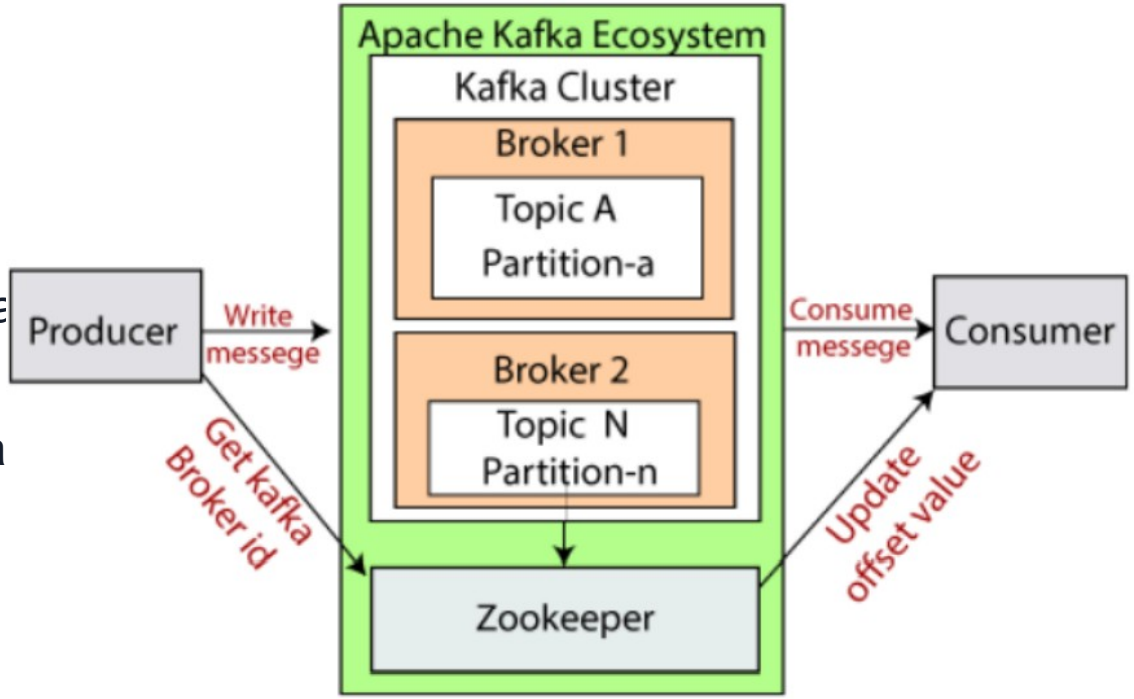
- **Kafka** can be **set up across multiple servers**, which are called **Kafka Brokers**.
- Mostly prefer to run **multiple Kafka broker instance** at the same time into our **Kafka cluster**.
- With multiple brokers, we can get the benefit of **data replication, fault tolerance**, and **high availability** of your Kafka cluster.
- **Kafka cluster** consists of multiple brokers to **maintain load balance.**
- Kafka brokers are **stateless**, so they use **ZooKeeper** for maintaining their cluster state.
- **Kafka broker instance** can handle hundreds of **thousands** of **reads** and **writes per second**.
- Kafka broker **leader election** can be done by **ZooKeeper**.



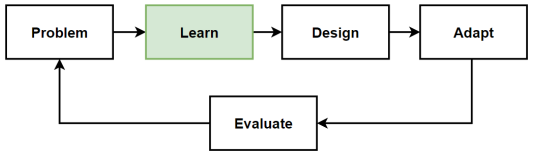
Apache Kafka Cluster Architecture: Zookeeper



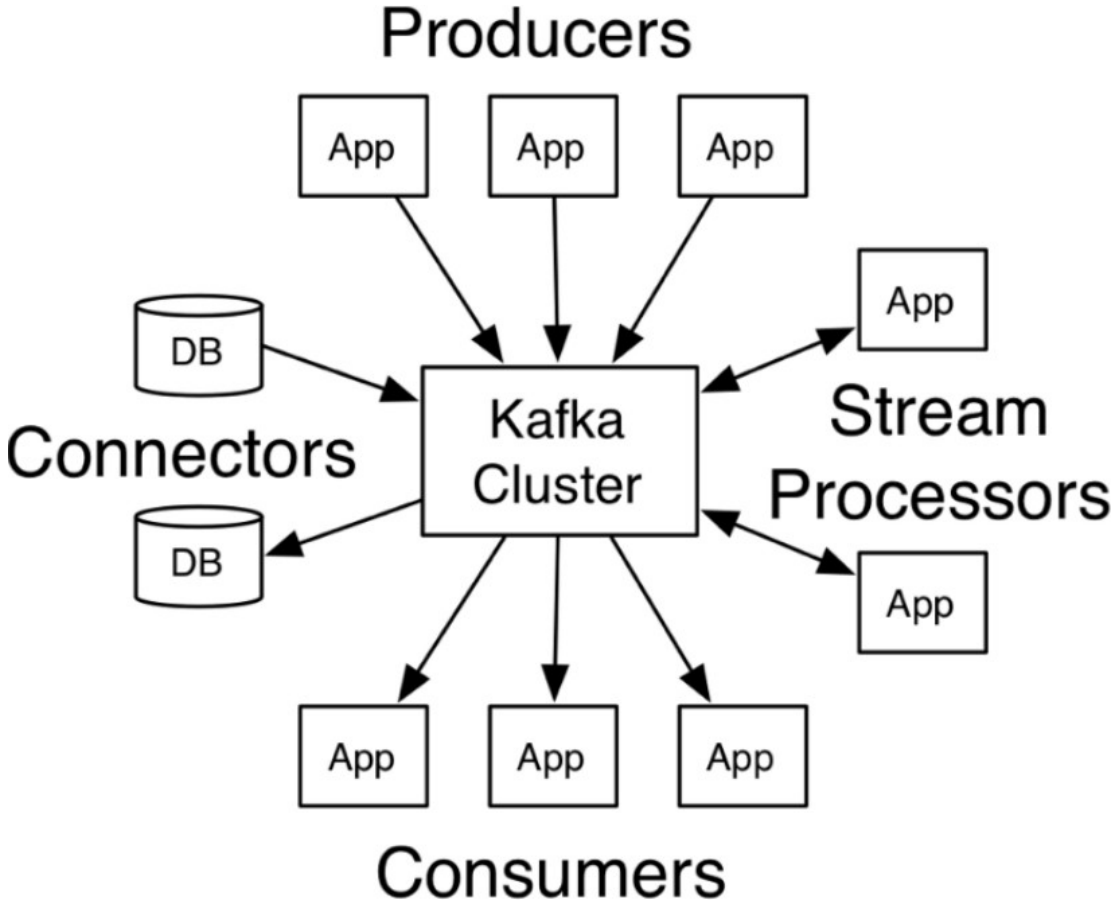
- **How we are managing and coordinating Kafka brokers ?**
- Kafka Cluster and including Kafka brokers need to manage.
- **Management job** is handled by the **master node** in the distributed systems.
- This master node **manage** and **maintenances** the **other worker(nodes)** in order to work correctly.
- In Apache Kafka, **there isn't any master** in the Apache Kafka Cluster.
- Apache Kafka Cluster isn't a master-worker architecture; it's a **master-less architecture**.
- **ZooKeeper** is used for **managing** and **coordinating Kafka broker.** The zookeeper manages and maintains the brokers in the cluster.
- Finds which brokers have been **crashed** or which broker recently **added** to the cluster and **manage** their **lifecycle**.



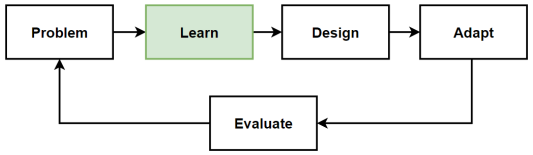
Apache Kafka Core APIs: Producer, Consumer, Streams and Connect API



- **Producers API**
  Producers push data to brokers. Publish a stream of data to Kafka topics. When the new broker is started, all the producers search and sends a message to that new broker.
- **Consumer API**
  Consumer API provide to subscribe to topics and process the streams of data. Manage messages have been consumed from Kafka by using partition offset. Consumer offset values are managed by ZooKeeper.
- **Streams API**
  Streams API that is useful for transforming data from one topic to another. Allows applications to act as a stream processor. Transforming the input streams to output streams.
- **Connect API**
  Provide to implement connectors that pulling data from external systems into Kafka. Use stream data between Kafka and other external systems. Collect metrics from servers into Kafka topics.

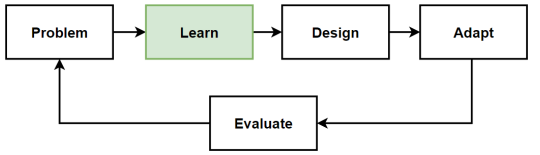


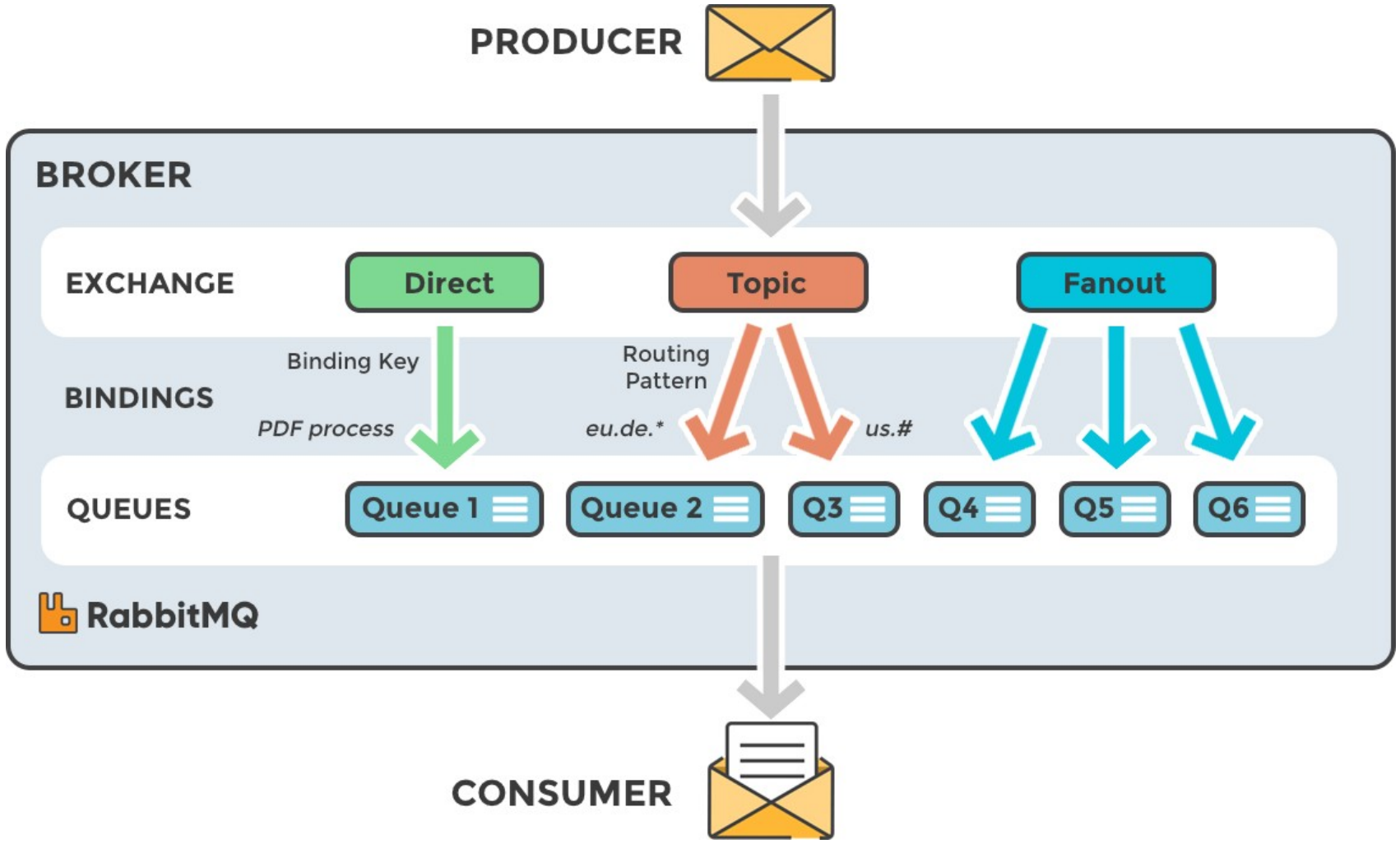
What is RabbitMQ ?



- **RabbitMQ** is a **message broker software** that implements the **Advanced Message Queuing Protocol (AMQP).**
- It allows applications to communicate with each other by **sending** and **receiving messages** through **queues**.
- **RabbitMQ** is a **message queuing system** that transmit a message received from any source to another source.
- Similar ones can be listed as **Apache Kafka, Msmq, Microsoft Azure Service Bus, Kestrel, ActiveMQ**.
- All **transactions** can be **listed in a queue** until the source to be transmitted gets up.
- It allows to **send** and **receive** messages **asynchronously**.
- **RabbitMQ's** support for **multiple operating systems** and open source code is one of the most preferred reasons.
- Main Components of RabbitMQ: **Producer**, **Queue**, **Consumer**, **Message**, **Exchange**, **Binding** and **FIFO**.

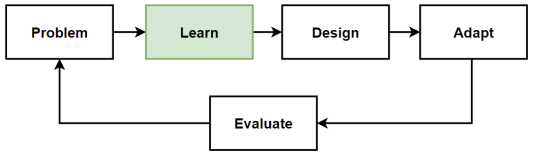
RabbitMQ Components: Producer, Queue, Consumer, Message, Exchange, Binding





https://www.cloudamqp.com/img/blog/exchanges-topic-fanout-direct.png

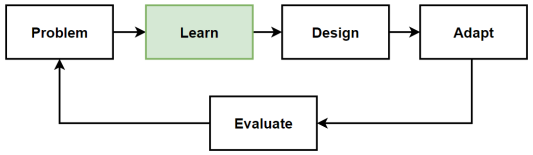
RabbitMQ Queue Properties



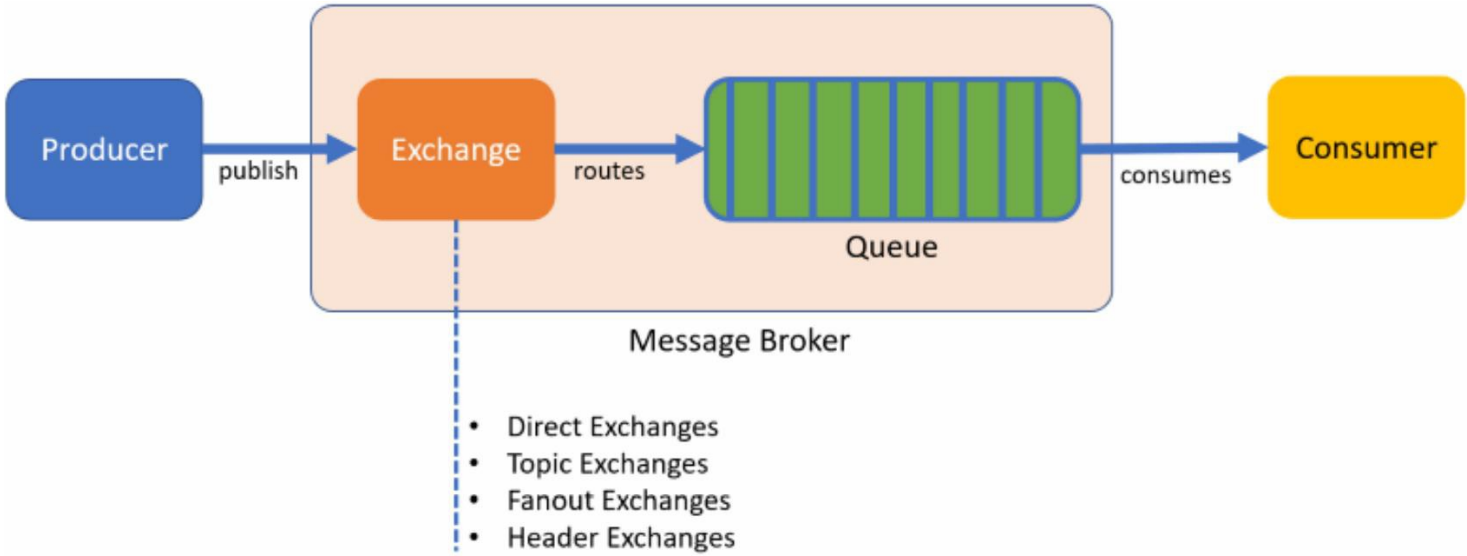
- **Queue Name**
  The name of the queue we have defined.
- **Durable**
  Determines the lifetime of the queue. If we want persistence, we have to set it true.
- **Exclusive**
  Contains information whether the queue will be used with other connections.
- **AutoDelete**
  Contains information about deletion of the queue with the data sent to the queue passes to the consumer side.

https://www.rabbitmq.com/tutorials/amqp-concepts.html

RabbitMQ Exchange Types

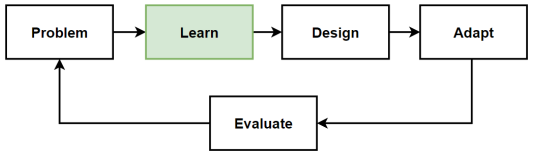


- RabbitMQ provides four types of exchanges: Direct, Fanout, Topic, and Headers.
- **Direct Exchange**
  Uses of a single queue is being addressed. The most appropriate queue is reached with the relevant direct exchange.
- **Topic Exchange**
  Messages are sent to different queues according to their subject. Incoming message is classified and sent to the related queue.
- Variation of the Publish / Subscribe pattern. Topic Exchange should be used to determine what kind of message they want to receive.

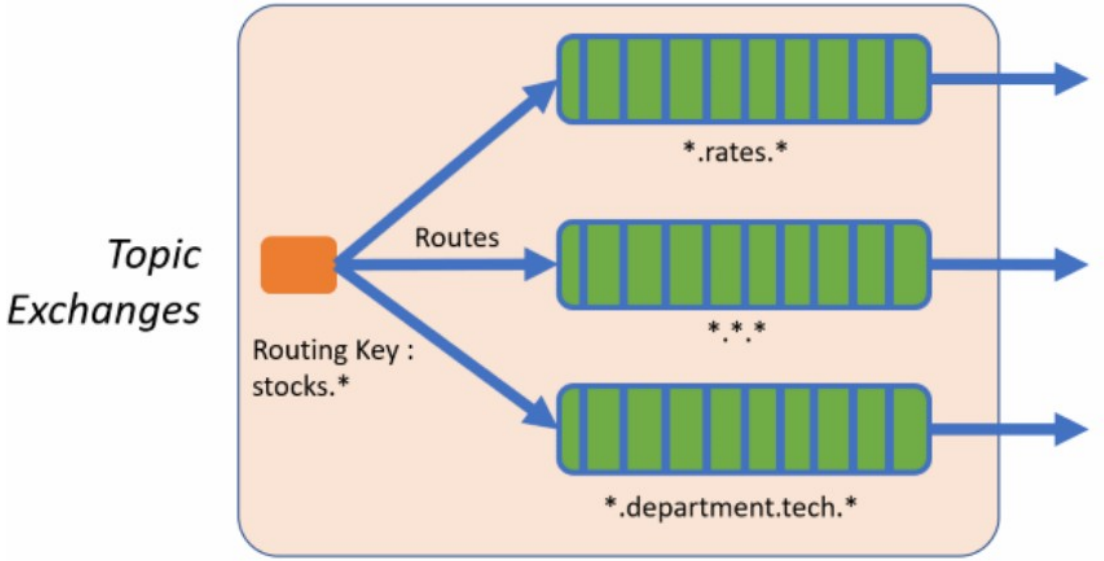


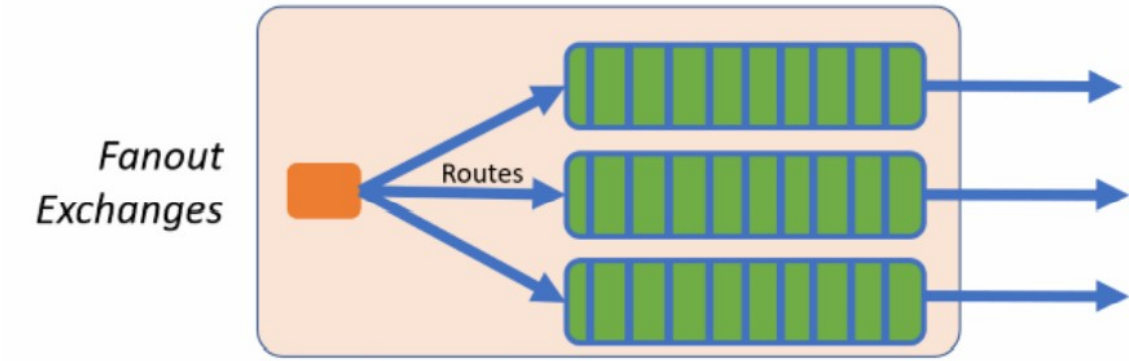
https://www.rabbitmq.com/tutorials/amqp-concepts.html

RabbitMQ Exchange Types-2



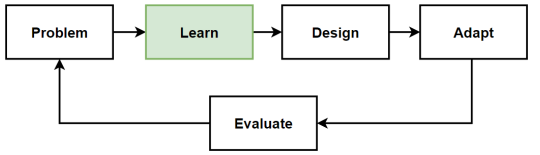
- **Fanout Exchange**
  Used in situations where the message should be sent to more than one queue. Especially applied in Broadcasting systems.
- Mainly used for games for global announcements.
- **Headers Exchange**
  Guided by the features added to the header of the message. Routing-Key used in other models is not used.
- Transmits to the correct queue with a few features and descriptions in message headers.
- The attributes on the header and the attributes on the queue must match each other's values.



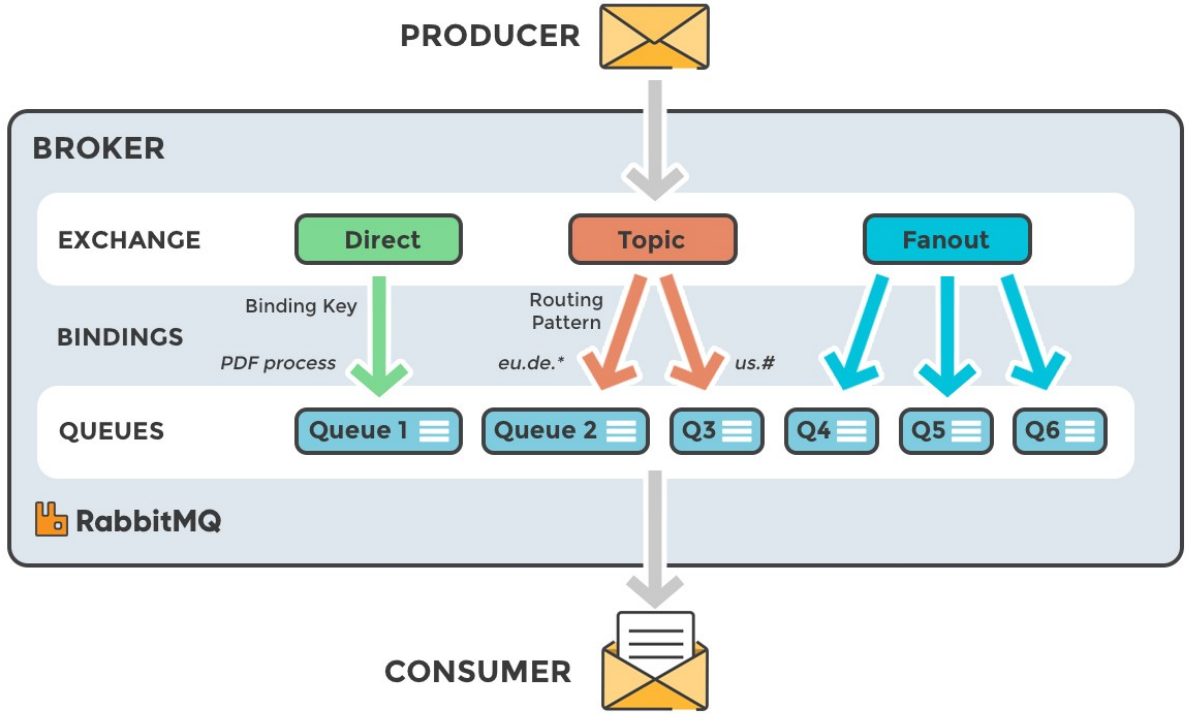


https://www.rabbitmq.com/tutorials/amqp-concepts.html

RabbitMQ Architecture



- The producer **never sends any messages directly** to a queue. Instead, the producer can only **send messages** to an **exchange**.
- **Exchange** has **2 sides**: it **receives messages** from **producers**, it **pushes** them to **queues**.
- Exchanges control the **routing of messages to queues** according to **exchange types**.
- RabbitMQ uses a **push model**.
- **Push-based queues** can overwhelm consumers if messages arrive faster than the consumers can process them.
- Each consumer can configure a **prefetch limit**.
- Push model is to **distribute messages individually** and quickly, and ensure that work is **concurrent way**.
- Messages are **processed in the same order** how they **arrived** in the **queue**.



https://www.cloudamqp.com/img/blog/exchanges-topic-fanout-direct.png

CÁM ƠN ĐÃ CHÚ Ý LẮNG NGHE!